In [1]:
```
import pandas as pd
```

In [8]:
```
import numpy as np
```

In [10]:
```
df = pd.read_csv('PlayTennis - PlayTennis.csv')
```

In [12]:
```
#df = pd.read_csv('c:\\Desktop\\NF\\PlayTennis - PlayTennis.csv')
```

In [13]:
```
df.head()
```

Out[13]:

	outlook	temp	humidity	windy	play
0	sunny	hot	high	False	no
1	sunny	hot	high	True	no
2	overcast	hot	high	False	yes
3	rainy	mild	high	False	yes
4	rainy	cool	normal	False	yes

In [14]:
```
df.describe
```

Out[14]:
```
<bound method NDFrame.describe of      outlook  temp humidity  windy play
0      sunny   hot     high  False   no
1      sunny   hot     high   True   no
2   overcast   hot     high  False  yes
3      rainy  mild     high  False  yes
4      rainy  cool   normal  False  yes
5      rainy  cool   normal   True   no
6   overcast  cool   normal   True  yes
7      sunny  mild     high  False   no
8      sunny  cool   normal  False  yes
9      rainy  mild   normal  False  yes
10     sunny  mild   normal   True  yes
11  overcast  mild     high   True  yes
12  overcast   hot   normal  False  yes
13     rainy  mild     high   True   no>
```

In [16]:
```
df.isnull().sum()
```

Out[16]:
```
outlook     0
temp        0
humidity    0
windy       0
play        0
dtype: int64
```

In [18]:
```
X=df.iloc[:,0:4]
```

In [19]:
```
Y= df.iloc[:,-1]
```

train tet split

In [27]:
```
from sklearn.model_selection import train_test_split
```

In [29]:
```
X_train,X_test,Y_train,Y_test= train_test_split(X,Y,test_size=0.2,random_state = 2)
```

model building

In [36]:
```
from sklearn.naive_bayes import GaussianNB
```

In [38]:
```
clf = GaussianNB()
```

In [39]:
```
clf.fit(X_train,Y_train)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[39], line 1
----> 1 clf.fit(X_train,Y_train)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:1151, in _fit_cont
ext.<locals>.decorator.<locals>.wrapper(estimator, *args, **kwargs)
   1144     estimator._validate_params()
   1146 with config_context(
   1147     skip_parameter_validation=(
   1148         prefer_skip_nested_validation or global_skip_validation
   1149     )
   1150 ):
-> 1151     return fit_method(estimator, *args, **kwargs)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\naive_bayes.py:263, in Gau
ssianNB.fit(self, X, y, sample_weight)
    240 """Fit Gaussian Naive Bayes according to X, y.
    241
    242 Parameters
   (...)
    260     Returns the instance itself.
    261 """
    262 y = self._validate_data(y=y)
--> 263 return self._partial_fit(
    264     X, y, np.unique(y), _refit=True, sample_weight=sample_weight
    265 )

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\naive_bayes.py:423, in Gau
ssianNB._partial_fit(self, X, y, classes, _refit, sample_weight)
    420     self.classes_ = None
    422 first_call = _check_partial_fit_first_call(self, classes)
--> 423 X, y = self._validate_data(X, y, reset=first_call)
    424 if sample_weight is not None:
    425     sample_weight = _check_sample_weight(sample_weight, X)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:621, in BaseEstima
tor._validate_data(self, X, y, reset, validate_separately, cast_to_ndarray, **chec
k_params)
    619         y = check_array(y, input_name="y", **check_y_params)
    620     else:
--> 621         X, y = check_X_y(X, y, **check_params)
    622     out = X, y
    624 if not no_val_X and check_params.get("ensure_2d", True):

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\utils\validation.py:1147,
in check_X_y(X, y, accept_sparse, accept_large_sparse, dtype, order, copy, force_a
ll_finite, ensure_2d, allow_nd, multi_output, ensure_min_samples, ensure_min_featu
res, y_numeric, estimator)
   1142         estimator_name = _check_estimator_name(estimator)
   1143     raise ValueError(
   1144         f"{estimator_name} requires y to be passed, but the target y is No
ne"
   1145     )
-> 1147 X = check_array(
   1148     X,
   1149     accept_sparse=accept_sparse,
   1150     accept_large_sparse=accept_large_sparse,
   1151     dtype=dtype,
   1152     order=order,
   1153     copy=copy,
   1154     force_all_finite=force_all_finite,
   1155     ensure_2d=ensure_2d,
   1156     allow_nd=allow_nd,
```

```
   1157     ensure_min_samples=ensure_min_samples,
   1158     ensure_min_features=ensure_min_features,
   1159     estimator=estimator,
   1160     input_name="X",
   1161 )
   1163 y = _check_y(y, multi_output=multi_output, y_numeric=y_numeric, estimator=
estimator)
   1165 check_consistent_length(X, y)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\utils\validation.py:838, i
n check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force
_all_finite, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimat
or, input_name)
    833 if pandas_requires_conversion:
    834     # pandas dataframe requires conversion earlier to handle extension dty
pes with
    835     # nans
    836     # Use the original dtype for conversion if dtype is None
    837     new_dtype = dtype_orig if dtype is None else dtype
--> 838     array = array.astype(new_dtype)
    839     # Since we converted here, we do not need to convert again later
    840     dtype = None

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:6324, in ND
Frame.astype(self, dtype, copy, errors)
   6317     results = [
   6318         self.iloc[:, i].astype(dtype, copy=copy)
   6319         for i in range(len(self.columns))
   6320     ]
   6322 else:
   6323     # else, only a single dtype is given
-> 6324     new_data = self._mgr.astype(dtype=dtype, copy=copy, errors=errors)
   6325     return self._constructor(new_data).__finalize__(self, method="astype")
   6327 # GH 33113: handle empty frame or series

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:
451, in BaseBlockManager.astype(self, dtype, copy, errors)
    448 elif using_copy_on_write():
    449     copy = False
--> 451 return self.apply(
    452     "astype",
    453     dtype=dtype,
    454     copy=copy,
    455     errors=errors,
    456     using_cow=using_copy_on_write(),
    457 )

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:
352, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    350         applied = b.apply(f, **kwargs)
    351     else:
--> 352         applied = getattr(b, f)(**kwargs)
    353     result_blocks = extend_blocks(applied, result_blocks)
    355 out = type(self).from_blocks(result_blocks, self.axes)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:51
1, in Block.astype(self, dtype, copy, errors, using_cow)
    491 """
    492 Coerce to the new dtype.
    493
   (...)
    507 Block
    508 """
    509 values = self.values
```

```
--> 511 new_values = astype_array_safe(values, dtype, copy=copy, errors=errors)
    513 new_values = maybe_coerce_values(new_values)
    515 refs = None

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:242,
in astype_array_safe(values, dtype, copy, errors)
    239     dtype = dtype.numpy_dtype
    241 try:
--> 242     new_values = astype_array(values, dtype, copy=copy)
    243 except (ValueError, TypeError):
    244     # e.g. _astype_nansafe can fail on object-dtype of strings
    245     #  trying to convert to float
    246     if errors == "ignore":

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:187,
in astype_array(values, dtype, copy)
    184     values = values.astype(dtype, copy=copy)
    186 else:
--> 187     values = _astype_nansafe(values, dtype, copy=copy)
    189 # in pandas we don't store numpy str dtypes, so convert to object
    190 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:138,
in _astype_nansafe(arr, dtype, copy, skipna)
    134     raise ValueError(msg)
    136 if copy or is_object_dtype(arr.dtype) or is_object_dtype(dtype):
    137     # Explicit copy, or required since NumPy can't view from / to object.
--> 138     return arr.astype(dtype, copy=True)
    140 return arr.astype(dtype, copy=copy)

ValueError: could not convert string to float: 'sunny'
```

Encoding

In [41]:
```
from sklearn.preprocessing import OneHotEncoder
```

In [47]:
```
ohe = OneHotEncoder(sparse_output = False , dtype = np.int32)
```

In [48]:
```
X_new = ohe.fit_transform(X)
```

In [49]:
```
from sklearn.model_selection import train_test_split
```

In [50]:
```
X_train,X_test,Y_train,Y_test= train_test_split(X,Y,test_size=0.2,random_state = 2)
```

In [51]:
```
from sklearn.preprocessing import OneHotEncoder
```

In [52]:
```
ohe = OneHotEncoder(sparse_output = False , dtype = np.int32)
```

In [53]:
```
X_new = ohe.fit_transform(X)
```

In [55]:
```
ohe = OneHotEncoder(drop="first",sparse_output=False ,dtype = np.int32)
```

Evaluation of model

In [56]:
```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, reca
```

In [61]:
```
y_pred = clf.predict(X_test)
```

```
---------------------------------------------------------------------------
ValueError                                Traceback (most recent call last)
Cell In[61], line 1
----> 1 y_pred = clf.predict(X_test)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\naive_bayes.py:101, in _BaseNB.predict(self, X)
     87 """
     88 Perform classification on an array of test vectors X.
     89
   (...)
     98     Predicted target values for X.
     99 """
    100 check_is_fitted(self)
--> 101 X = self._check_X(X)
    102 jll = self._joint_log_likelihood(X)
    103 return self.classes_[np.argmax(jll, axis=1)]

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\naive_bayes.py:269, in GaussianNB._check_X(self, X)
    267 def _check_X(self, X):
    268     """Validate X, used only in predict* methods."""
--> 269     return self._validate_data(X, reset=False)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\base.py:604, in BaseEstimator._validate_data(self, X, y, reset, validate_separately, cast_to_ndarray, **check_params)
    602         out = X, y
    603 elif not no_val_X and no_val_y:
--> 604     out = check_array(X, input_name="X", **check_params)
    605 elif no_val_X and not no_val_y:
    606     out = _check_y(y, **check_params)

File C:\ProgramData\anaconda3\Lib\site-packages\sklearn\utils\validation.py:838, in check_array(array, accept_sparse, accept_large_sparse, dtype, order, copy, force_all_finite, ensure_2d, allow_nd, ensure_min_samples, ensure_min_features, estimator, input_name)
    833 if pandas_requires_conversion:
    834     # pandas dataframe requires conversion earlier to handle extension dtypes with
    835     # nans
    836     # Use the original dtype for conversion if dtype is None
    837     new_dtype = dtype_orig if dtype is None else dtype
--> 838     array = array.astype(new_dtype)
    839     # Since we converted here, we do not need to convert again later
    840     dtype = None

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\generic.py:6324, in NDFrame.astype(self, dtype, copy, errors)
   6317     results = [
   6318         self.iloc[:, i].astype(dtype, copy=copy)
   6319         for i in range(len(self.columns))
   6320     ]
   6322 else:
   6323     # else, only a single dtype is given
-> 6324     new_data = self._mgr.astype(dtype=dtype, copy=copy, errors=errors)
   6325     return self._constructor(new_data).__finalize__(self, method="astype")
   6327 # GH 33113: handle empty frame or series

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:451, in BaseBlockManager.astype(self, dtype, copy, errors)
    448 elif using_copy_on_write():
    449     copy = False
--> 451 return self.apply(
```

```
    452     "astype",
    453     dtype=dtype,
    454     copy=copy,
    455     errors=errors,
    456     using_cow=using_copy_on_write(),
    457 )

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\managers.py:
352, in BaseBlockManager.apply(self, f, align_keys, **kwargs)
    350         applied = b.apply(f, **kwargs)
    351     else:
--> 352         applied = getattr(b, f)(**kwargs)
    353     result_blocks = extend_blocks(applied, result_blocks)
    355 out = type(self).from_blocks(result_blocks, self.axes)

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\internals\blocks.py:51
1, in Block.astype(self, dtype, copy, errors, using_cow)
    491 """
    492 Coerce to the new dtype.
    493
   (...)
    507 Block
    508 """
    509 values = self.values
--> 511 new_values = astype_array_safe(values, dtype, copy=copy, errors=errors)
    513 new_values = maybe_coerce_values(new_values)
    515 refs = None

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:242,
in astype_array_safe(values, dtype, copy, errors)
    239     dtype = dtype.numpy_dtype
    241 try:
--> 242     new_values = astype_array(values, dtype, copy=copy)
    243 except (ValueError, TypeError):
    244     # e.g. _astype_nansafe can fail on object-dtype of strings
    245     #  trying to convert to float
    246     if errors == "ignore":

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:187,
in astype_array(values, dtype, copy)
    184     values = values.astype(dtype, copy=copy)
    186 else:
--> 187     values = _astype_nansafe(values, dtype, copy=copy)
    189 # in pandas we don't store numpy str dtypes, so convert to object
    190 if isinstance(dtype, np.dtype) and issubclass(values.dtype.type, str):

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\core\dtypes\astype.py:138,
in _astype_nansafe(arr, dtype, copy, skipna)
    134     raise ValueError(msg)
    136 if copy or is_object_dtype(arr.dtype) or is_object_dtype(dtype):
    137     # Explicit copy, or required since NumPy can't view from / to object.
--> 138     return arr.astype(dtype, copy=True)
    140 return arr.astype(dtype, copy=copy)

ValueError: could not convert string to float: 'overcast'
```

In [62]:

```
from sklearn.preprocessing import OneHotEncoder

ohe = OneHotEncoder(drop='first', sparse_output=False)

X_new = ohe.fit_transform(X)
```

In [63]:

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, Y_train, Y_test = train_test_split(
    X_new, Y, test_size=0.2, random_state=2
)
```

In [64]:
```
from sklearn.naive_bayes import GaussianNB

clf = GaussianNB()

clf.fit(X_train, Y_train)
```

Out[64]:
```
▾ GaussianNB
GaussianNB()
```

In [65]:
```
y_pred = clf.predict(X_test)
```

In [ ]:
```

```